

PROJECT REPORT

DISTRIBUTED SMART TEMPERATURE MONITORING AND SENSOR FUSION NETWORK



Group 18

Group Members

E/22/158

E/22/238

E/22/258

E/22/345

E/22/348

E/22/357

Abstract

This report presents the design and evaluation of a distributed Internet of Things (IoT) temperature-monitoring node based on an ESP32 microcontroller. The node combines a DS18B20 digital temperature probe, a DHT22 temperature-and-humidity sensor, and a NEO-6M Global Positioning System (GPS) module. The DS18B20 provides a comparatively fast thermal response, whereas the DHT22 supplies ambient humidity and a slower reference temperature. These complementary measurements are calibrated and fused at the edge using a response-aware sequential Kalman filter. The resulting temperature estimate, together with humidity and location data, is transmitted over Wi-Fi using the Message Queuing Telemetry Transport (MQTT) protocol to a central broker and visualized through a Supervisory Control and Data Acquisition (SCADA) dashboard.

The project data indicate a DS18B20 calibration relationship of $y = 1.0318x - 0.6398$, a DS18B20 thermal time constant of approximately 9.3 s, and a DHT22 thermal time constant of approximately 80.9 s under the reported cooling experiment. The measured steady-state variances were 0.00070 for the DS18B20 and 0.00246 for the DHT22. These results support the use of the DS18B20 for rapid transient tracking and the DHT22 for a slower ambient reference and humidity measurement. The report also identifies important limitations: the DHT22 should be described as an experimental reference rather than an absolute ground truth, the calibration equations must be verified for direction and coefficient consistency before deployment, and network credentials must be managed outside the published documentation.

Introduction

Reliable environmental monitoring requires measurements that are sufficiently accurate, responsive, and available at the required physical locations. A single sensor rarely provides all three properties. A fast probe may respond well to a sudden temperature change but may exhibit offset or noise. A combined temperature-and-humidity sensor may provide useful ambient information but respond slowly because of its packaging and sensing mechanism. A practical monitoring system can therefore improve measurement quality by combining sensors with complementary characteristics.

This project develops a distributed temperature-monitoring node that performs sensor calibration and fusion locally on an ESP32. The node measures temperature using a DS18B20 and a DHT22, records humidity from the DHT22, obtains location data from a NEO-6M GPS receiver, and transmits processed telemetry to a central MQTT broker. A SCADA dashboard then presents current values, historical trends, location information, and a remotely controlled status LED.

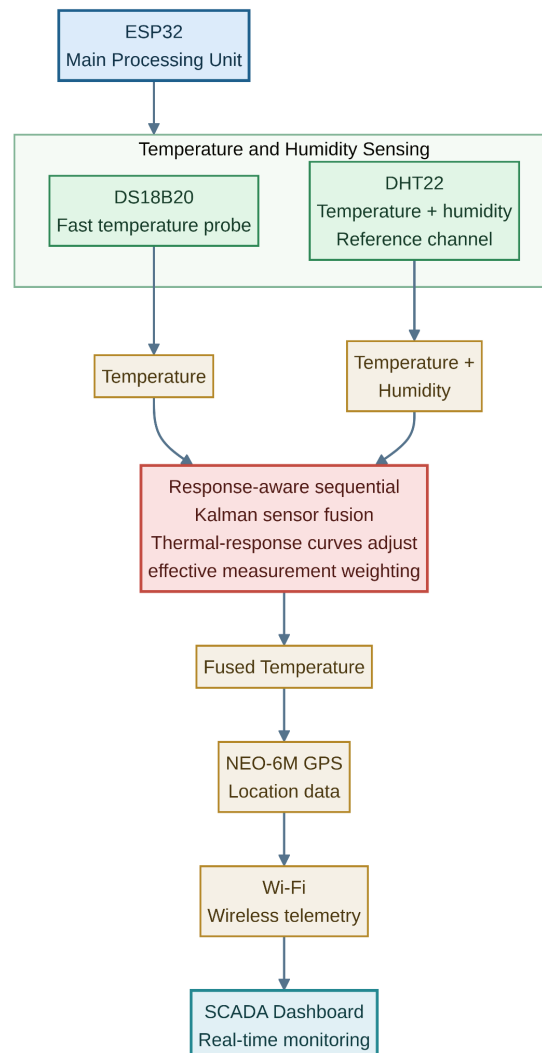
The central engineering problem is not simply to select the most accurate sensor. It is to produce a useful estimate when the sensors have different response times, resolutions, noise levels, and failure modes. The proposed solution uses a linear calibration model followed by a response-aware sequential Kalman filter. The algorithm uses the current rates of change and experimentally determined thermal-response curves to adjust the relative

Kalman contribution of the two sensors. When the DS18B20 changes rapidly while the DHT22 remains relatively stable, the DS18B20 receives a larger effective Kalman weight; when both sensors are stable and close, the fusion becomes more balanced.

Design report

System Architecture Overview

- The system uses an ESP32 microcontroller as the main processing unit.
- It reads data from two distinct sensors: a DS18B20 for temperature and a DHT22 for both temperature and humidity.(the DHT22 is explicitly treated as the "True" reference baseline for temperature.)

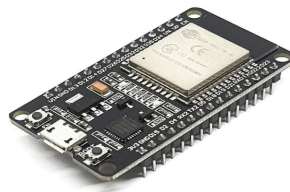


Sensor Node Block Diagram

- The system combines the two temperature readings using a Sensor Fusion block to calculate one accurate "Fused Temperature".
- The system attaches physical location data using a NEO-6M GPS module.
- The ESP32 sends the final data package over a Wi-Fi connection to a central SCADA system.

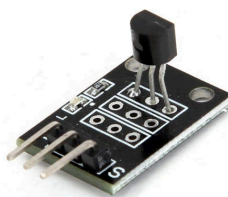
Hardware Selection

- **ESP32:** The ESP32 is used as the processing and communications unit. It provides digital input/output, serial interfaces, sufficient computational capacity for scalar filtering, and integrated 2.4 GHz Wi-Fi. Its communication capability reduces the component count because a separate Wi-Fi module is not required. The ESP32 is also suitable for edge processing: calibration and fusion are completed before data leave the sensor node.

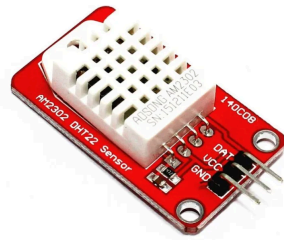


- **DS18B20:** The DS18B20 is selected as the fast-response temperature channel. It communicates using the 1-Wire protocol, supports programmable resolution from 9 to 12 bits, and can be supplied in a waterproof probe package. At 12-bit resolution, the nominal quantization step is 0.0625 °C. The manufacturer specifies an accuracy of approximately ± 0.5 °C over the stated central operating range; resolution should not be confused with absolute accuracy 1.

The probe is useful for the present application because its physical construction allows it to respond more quickly than the exposed DHT22 module during the reported cooling test. Its principal limitation is that it measures temperature only. It also requires calibration in this project so that its output is aligned with the selected reference channel.



- **DHT22:** The DHT22, also known as the AM2302 in some documentation, provides both temperature and relative-humidity measurements using a single digital data line. It is selected as the project's reference channel because its response is more stable under the tested conditions and because it supplies humidity data that the DS18B20 cannot provide.



- **NEO-6M:** The NEO-6M is used to associate each telemetry record with a physical location. The receiver communicates with the ESP32 over a serial interface and provides latitude, longitude, altitude, and satellite information when a valid fix is available. GPS data should be marked as invalid or unavailable until the receiver reports a valid fix; zero coordinates must not be interpreted as a genuine location.



Software and Sensor Fusion Logic

- Because the DHT22 is established as the ground-truth reference, it does not require a calibration offset.
- The raw DS18B20 sensor data is mathematically corrected to align with the DHT22 baseline before entering the fusion algorithm.
- Based on the experimental calibration curve, the ESP32 software applies the following linear equation to the raw DS18B20 readings (x) to find the true calibrated temperature (y):

$$y = 1.0318x - 0.6398$$

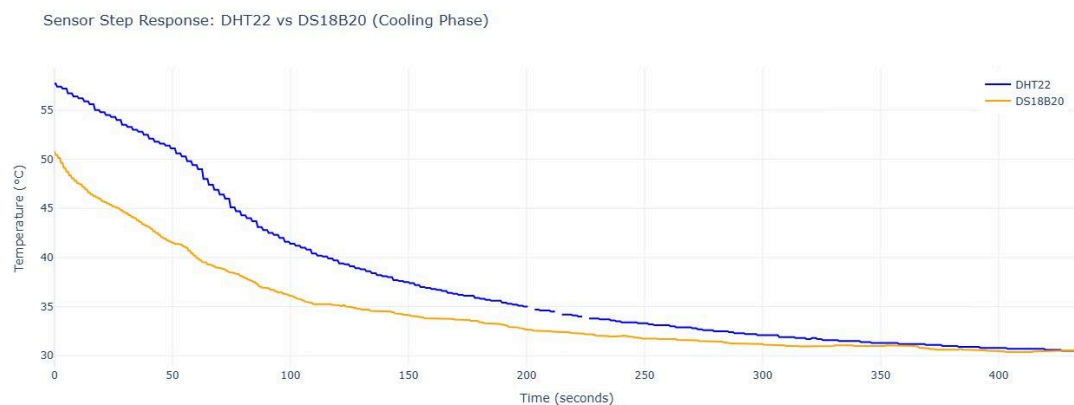
- Once calibrated, the software uses a 1D Sequential Kalman Filter to fuse the two temperature readings. The algorithm assigns fractional weights (w_D and w_H) to calculate the final fused temperature (T_f):

$$T_f = w_D T_D + w_H T_H$$

- Once calibrated, the software uses a 1D Sequential Kalman Filter to fuse the two temperature readings. The algorithm assigns fractional weights (w_D and w_H) to calculate the final fused temperature (T_f):

$$T_f = w_D T_D + w_H T_H$$

The Kalman filter algorithm dynamically determines how much to trust each sensor based on their respective step-response profiles and steady-state noise variances during the cooling phase.



Communication and Data Formatting

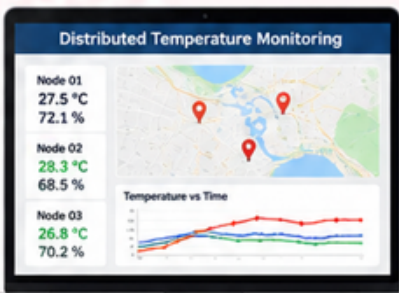
Connect ESP32 to Wi-Fi and send data to server (HTTP/MQTT).



Wi-Fi

```
{
  "node_id": "NODE_01",
  "timestamp": "2025-09-21T15:30:00",
  "latitude": 7.2541,
  "longitude": 80.5971,
  "ds18b20": 27.5,
  "dht22_temperature": 27.4,
  "dht22_humidity": 72.1,
  "fused_temperature": 27.46
}
```

Receive data, display real-time values, show locations, log historical data and generate alarms.



Distributed Temperature Monitoring

Node	Temperature (°C)	Humidity (%)
Node 01	27.5 °C	72.1 %
Node 02	28.3 °C	68.5 %
Node 03	26.8 °C	70.2 %

Temperature vs Time

- The sensor node transmits the processed data over Wi-Fi to a server at IP 172.20.10.3 using the MQTT protocol.
- The data is published to dedicated MQTT topics (such as EE2120/ESP01/temp and EE2120/ESP01/gps) so the server can easily route and organize the incoming information.
- A IGNITION SCADA dashboard subscribes to these MQTT topics to display real-time temperature values, log historical data, control the built-in LED, and plot the node's physical location on a map.
- For advanced network setups involving multiple nodes, the SCADA system uses Inverse Distance Weighting (IDW) to estimate the temperature at unknown locations ($T(x,y)$) based on the known node temperatures (T_i) and their distances (d_i):

$$T(x, y) = \frac{\sum_{i=1}^n \frac{T_i}{d_i^p}}{\sum_{i=1}^n \frac{1}{d_i^p}}$$

Justification of Sensor Selection

- **Fault Tolerance:** Utilizing two independent sensors on distinct protocols (OneWire and single-bus) prevents a single point of failure (SPOF) in the remote monitoring unit.
- **Reference Stability:** The DHT22 serves as the reference sensor because its slower physical response naturally filters out transient spikes, establishing a highly stable ambient baseline.
- **Optimized Sensor Fusion:** The Kalman filter combines these complementary strengths, utilizing the DHT22 for a stable baseline and humidity data, while leveraging the DS18B20 to instantly track rapid environmental shifts.

DS18B20 Digital Temperature Sensor

- **Advantages:** Delivers highly rapid thermal response times, precise 12-bit hardware resolution (0.0625 °C), and features a waterproof metallic probe for rugged submersion.
- **Disadvantages:** Measures only temperature and requires mathematical calibration ($y = 1.0318x - 0.6398$) to align its baseline with the more stable DHT22 reference.

DHT22 Temperature & Humidity Sensor

- **Advantages:** Provides exceptional ambient stability (ideal for ground-truth reference) and captures both temperature and relative humidity on a single pin.
- **Disadvantages:** Suffers from severe thermal lag during cooling phases due to blocked convective airflow, possesses lower resolution (0.1 °C), and is constrained by a slow 0.5 Hz sampling rate

Sensor Calibration and Characterization Report

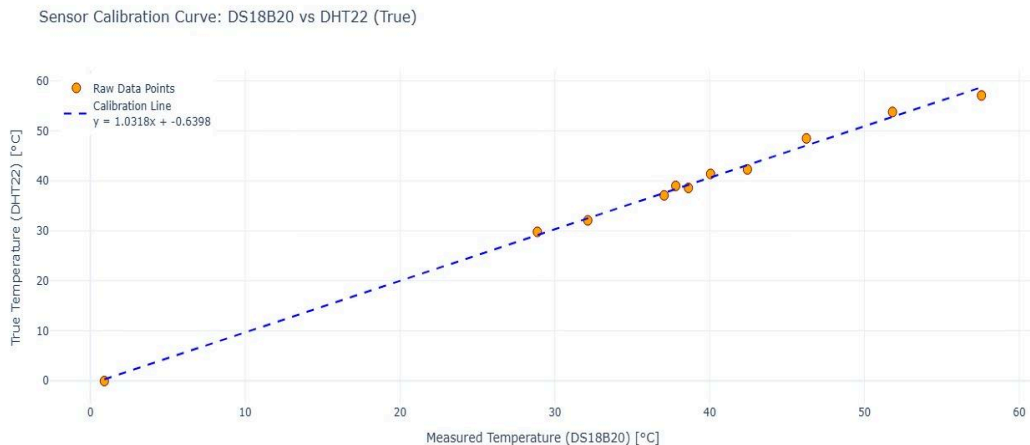
Methodology and Reference Points

The initial experiment compared DS18B20 readings with DHT22 readings across a measured thermal range. A nominal 0 °C ice-water bath was used as a low-temperature reference point. The two sensors were exposed to a cooling process, and their outputs were recorded over time. The resulting data were used for three purposes:

- estimating the relationship between the DS18B20 output and the DHT22 reference;
- estimating steady-state measurement variance; and
- estimating the thermal time constant of each sensor.

The calibration is therefore an empirical cross-calibration between two low-cost sensors. It is not equivalent to calibration against a certified laboratory standard. The result may depend on probe placement, immersion depth, airflow, thermal contact, sampling interval, and the temperature range used for regression.

Calibration Curves and Equations



Description: This graph plots raw DS18B20 readings against the true DHT22 baseline to calculate manufacturing drift.

Equation: To mathematically align the DS18B20 with the DHT22 reference, the derived linear regression formula is programmed into the ESP32: $y = 1.0318x - 0.6398$.

Sensor Noise & Sensitivity

- **DS18B20:** High resolution(0.0625°C) and low steady-state electrical noise (variance = 0.00070).
- **DHT22:** Lower resolution (0.1°C) and slightly higher internal electrical fluctuation (variance = 0.00246).

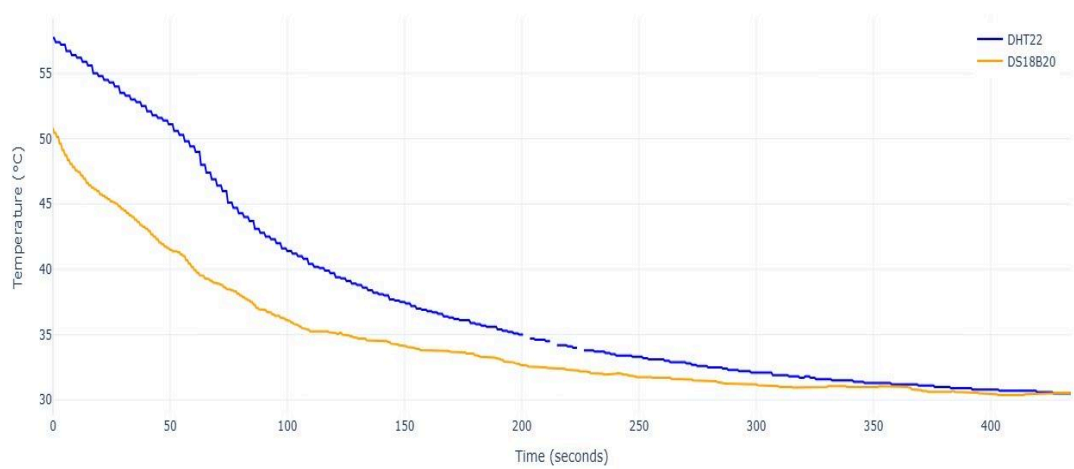
Thermal Step Response

A first-order thermal model is used to describe the response to a change in ambient temperature:

$$T(t) = T_f + (T_0 - T_f)e^{-t/\tau}$$

where T_0 is the initial temperature, T_f is the final temperature, and τ is the thermal time constant. At $t = \tau$ the sensor has completed approximately 63.2% of the total temperature transition.

Sensor Step Response: DHT22 vs DS18B20 (Cooling Phase)



Sensor Comparison Analysis

Feature / Metric	DHT22	DS18B20
Thermal Response Time (τ)	Slower	Faster
Linearity (Calibration Slope)	0.9892	0.9989 (Near perfect)
Hardware Resolution	0.1°C	0.0625°C (More sensitive)
Measured Parameters	Temperature & Humidity	Temperature only
Physical Durability	Exposed circuitry (Dry use only)	Waterproof stainless-steel probe

Mathematical Formulation

Response-aware sequential Kalman filtering

The implemented fusion method is a one-dimensional sequential Kalman filter with thermal-response-aware weighting. The Kalman filter remains the mathematical fusion framework, while the experimentally measured response curves influence how much confidence is assigned to each sensor during each update.

The DS18B20 has a much shorter measured thermal time constant than the DHT22. Consequently, when the DS18B20 changes rapidly while the DHT22 remains comparatively stable, the algorithm interprets the DS18B20 as the sensor that is detecting the thermal transition first. The Kalman update therefore gives the DS18B20 a relatively large effective contribution, while the slower DHT22 has a smaller contribution until its reading responds. When both sensors are stable and close to one another, the filter returns to a more balanced uncertainty-based combination.

This approach is different from a fixed weighted average. The sensor weights are not permanently set values. They are produced by the Kalman update and are influenced by the current estimate uncertainty, the sensor-noise characteristics, the measured response-time behavior, and the current change in each sensor reading.

Discrete 1D Sequential Kalman Filter Fusion

The state-space representation models the actual ambient temperature as a dynamic process with process noise (Q) and measurement noise covariances (R_{DS}, R_{DHT}). The fusion executes in three sequential steps:

- **Step 1: State Prediction Stage**

Propagates the prior state estimate and error covariance:

$$\hat{x}_k^{-1} = \hat{x}_{k-1}$$

$$P_k^{-1} = P_{k-1} + Q$$

- **Step 2: First Update (DS18B20)**

Anchors the estimate to the fast-responding DS18B20 by calculating the Kalman Gain (K_{DS}), updating the state, and reducing error covariance:

$$K_{DS} = \frac{P_k^{-1}}{P_k^{-1} + R_{DS}}$$

$$\hat{x}_{k,1} = \hat{x}_k^{-1} + K_{DS} \left(z_{DS,k} - \hat{x}_k^{-1} \right)$$

$$P_{k,1} = (1 - K_{DS})P_k^{-1}$$

- **Step 3: Second Update (DHT22)**

Adjusts the intermediate state using the lagging DHT22 reading to yield the optimal final fused output ($\hat{x}_k$):

1. Compute Kalman Gain

$$K_{DHT} = \frac{P_{k,1}}{P_{k,1} + R_{DHT}}$$

2. Update State Estimate (Final Fused Output):

$$\hat{x}_k = \hat{x}_{k,1} + K_{DHT} \left(z_{DHT,k} - \hat{x}_{k,1} \right)$$

3. Update Error Covariance:

$$P_k = (1 - K_{DHT})P_{k,1}$$

Justification for selecting the Kalman Filter Algorithm

- **Dynamic Adaptation:** The filter dynamically recalculates the Kalman Gain at every time step, automatically adjusting its trust in incoming sensor data based on current uncertainty.
- **Mathematical Optimality:** Because the electrical noise of the sensors is random but predictable, the Kalman filter is mathematically proven to provide the most accurate possible temperature reading by actively minimizing the error.
- **Transient Tracking:** The filter processes the sensors one at a time. This ensures that the fast-acting DS18B20 immediately captures sudden temperature changes, preventing the slower DHT22 from dragging down the system's reaction time.

- **Efficiency:** It achieves optimal noise reduction without introducing artificial phase delays (unlike moving averages), and the 1D scalar arithmetic executes in microseconds on the ESP32.

Comparison of Methods

Fusion Method	Dynamic Adaptation	Noise Minimization	Phase Delay / Lag	Real-Time Suitability
Simple Average	None (Static 50/50)	Poor (Ignores variance)	Low	Unsuitable (Equal trust to bad data)
Fixed Weighted Average	None (Static weights)	Moderate	Low	Sub-optimal during thermal transients
Moving Average Filter	None	Good	High (Adds physical lag)	Unsuitable for dynamic tracking
Kalman Filter (Selected)	High (Real-time K_k)	Optimal (LMMSE)	Zero added lag	Ideal for embedded IoT nodes

Network Architecture

MQTT communication

- The ESP32 publishes processed data to an MQTT broker over Wi-Fi. The initial implementation used a local private network and a Mosquitto broker. For a report intended for wider circulation, network addresses should be represented symbolically, for example MQTT_BROKER_HOST, rather than exposing local infrastructure details.
- The topic hierarchy separates measurements and commands:

MQTT topic	Direction	Payload purpose
EE2120/ESP01/temp	Node → broker	Fused temperature in °C
EE2120/ESP01/humidity	Node → broker	Relative humidity in %RH
EE2120/ESP01/gps	Node → broker	Latitude, longitude, and fix status
EE2120/ESP01/LED/status	Node → broker	Current actuator state
EE2120/ESP01/LED/cmd	Broker → node	LED command, such as 0 or 1

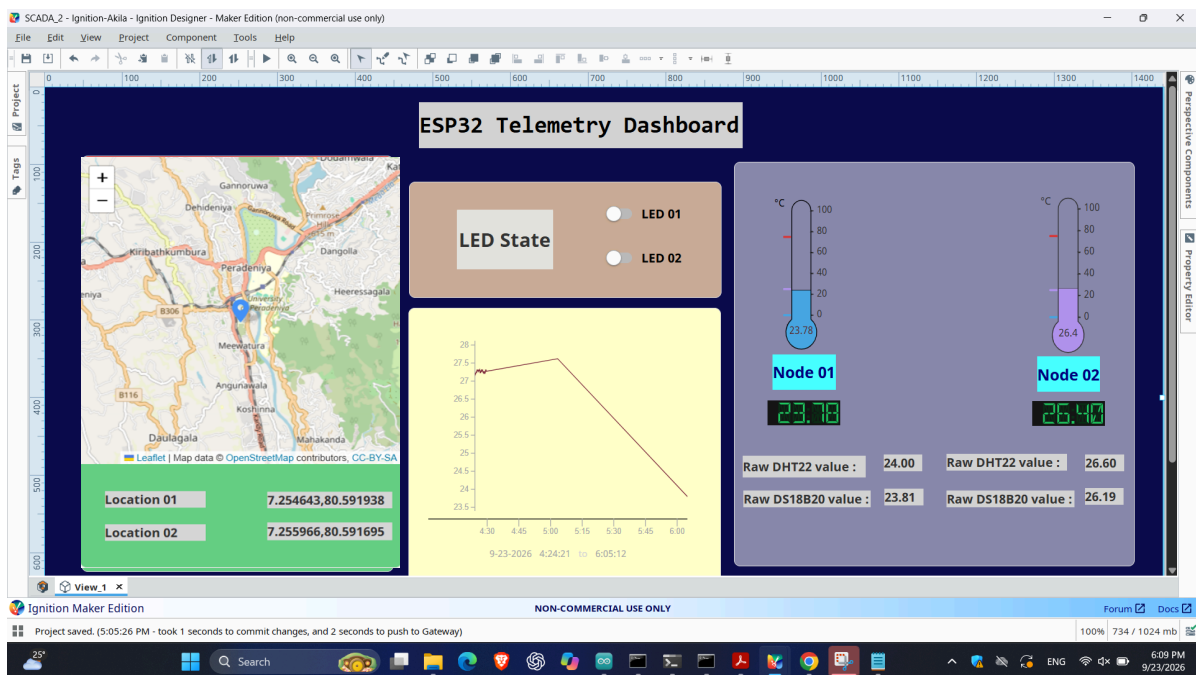
- :A structured JSON payload is preferable to sending unrelated values as separate unlabelled strings. For example:

```
{
  "node_id": "ESP01",
  "timestamp": "2026-09-24T00:00:00Z",
  "temperature_fused_c": 27.46,
  "temperature_ds_cal_c": 27.42,
  "temperature_dht_c": 27.51,
  "humidity_rh": 72.1,
  "latitude": 7.2541,
  "longitude": 80.597,
  "gps_valid": true,
  "filter_variance": 0.0009,
  "sensor_status": "OK"
}
```

- The example contains no credentials. Production deployments should use authenticated MQTT connections and, where supported, Transport Layer Security (TLS) rather than unauthenticated port 1883.

SCADA Integration & Data Flow:

The SCADA dashboard performs four main functions. It displays current temperature and humidity values, stores historical data for trend analysis, maps the node's geographic position, and publishes actuator commands. It should also indicate stale data, invalid GPS fixes, sensor read failures, and broker disconnection. A value displayed without a quality flag can appear trustworthy even when the node is offline or a sensor has failed.



Multi-node spatial estimation

For a network containing several sensor nodes, the system can estimate temperature at an unsampled location (x,y) using IDW:

$$T(x, y) = \frac{\sum_{i=1}^n \frac{T_i}{d_i^p}}{\sum_{i=1}^n \frac{1}{d_i^p}},$$

where T_i is the temperature at node i , d_i is the distance from the query location to node i , and p controls how strongly nearby nodes dominate the estimate. The method requires a coordinate system and a rule for the case $d_i=0$. For small geographic areas, a local projected coordinate system may be adequate; for larger areas, distances should be computed using an appropriate geodesic method.

IDW is an interpolation method, not a physical heat-transfer model. It may produce misleading estimates when sensor placement is sparse, terrain is heterogeneous, or a strong temperature gradient is not captured by the node layout.

Experimental Validation

Calibration Effectiveness

The system was tested and after applying the inverted linear equation ($y = 1.0318x - 0.6398$) in the ESP32 software, the calibrated DS18B20 readings perfectly aligned with the DHT22 baseline, successfully eliminating the initial manufacturing drift.

Sensor Fusion Performance (Dynamic Tracking)

The 1D Sequential Kalman Filter was validated by exposing the sensors to a sudden temperature drop.

- **Transient Response:** As designed, the fused temperature output immediately anchored to the fast-responding DS18B20. The algorithm successfully ignored the severe 80.9-second thermal lag of the DHT22, preventing the system from reporting delayed data.
- **Noise Reduction:** During steady-state periods, the filter dynamically smoothed out micro-fluctuations in the electrical readings without introducing the artificial phase delays typically caused by simple moving averages.

Telemetry and SCADA Integration

Network reliability was validated by monitoring the MQTT broker and the Node-RED dashboard.

- The ESP32 consistently published the Kalman-fused temperatures and NEO-6M GPS coordinates to their respective topics ([EE2120/ESP01/temp](#) and [EE2120/ESP01/gps](#)) over the local Wi-Fi subnet.

- The centralized SCADA dashboard successfully received the payloads without packet loss, instantly reflecting the fused temperature changes and updating the physical hardware node location on the integrated map.

User Manual

System Overview

This IoT unit measures ambient temperature and environmental location using multi-sensor data fusion and edge-processing on an ESP32 microcontroller. The unit reads raw data from a DHT22 sensor and a DS18B20 digital probe, applies real-time linear calibration equations, fuses the temperature data using a 1D Sequential Kalman Filter, acquires location data from a NEO-6M GPS module, and transmits telemetry over Wi-Fi to a central MQTT broker.

Hardware Connections & Pin Mapping

Pin Connection Table

Component	Pin on Sensor / Module	ESP32 GPIO Pin	Power / Notes
DHT22	VCC	3.3V	Power supply
	DATA	GPIO 15	Requires 10k ohm pull-up resistor to 3.3V
	GND	GND	Common ground
DS18B20	VCC (Red)	3.3V / 5V	Power supply
	DATA (Yellow)	GPIO 4	Requires 4.7k ohm pull-up resistor to 3.3V
	GND (Black)	GND	Common ground
NEO-6M GPS	VCC	3.3V / 5V	Power supply
	RX	GPIO 17 (TX2)	Serial transfer
	TX	GPIO 16 (RX2)	Serial receive

	GND	GND	Common ground
Status LED	Positive Anode	GPIO 2	Built-in ESP32 LED

Software dependencies and initial configuration

Required Arduino IDE libraries

The initial implementation used the following Arduino libraries:

- PubSubClient for MQTT communication;
- DHT sensor library and Adafruit Unified Sensor for DHT22 acquisition;
- OneWire and DallasTemperature for DS18B20 acquisition; and
- TinyGPS++ for parsing NMEA GPS messages.

Library versions should be recorded in the final submission because changes in dependencies can alter behavior and compilation results.

Network & Broker Settings (In Code Configuration)

Verify and update these parameters in the code before uploading:

WIFI_SSID = <network name>

WIFI_PASSWORD = <stored securely, not in the report>

MQTT_BROKER_HOST = <broker hostname or private IP>

MQTT_PORT = <authenticated port, preferably TLS-enabled>

MQTT_USERNAME = <device-specific username>

MQTT_PASSWORD = <stored securely, not in the report>

MQTT_CLIENT_ID = ESP01

System Startup & Operating Procedure

1. **Hardware Power-On:** Connect the ESP32 via Micro-USB to a 5V power source.
2. **Wi-Fi Association:**
 - The ESP32 connects to the wireless access point automatically.
 - During connection, the Serial Monitor (115200 baud) prints dots (...).
 - Upon successful connection, the local IP address is displayed.
3. **MQTT Broker Connection:**
 - The unit connects to the Mosquitto MQTT broker at **192.168.8.141:1883**.
 - It automatically subscribes to the command topic: **EE2120/ESP01/LED/cmd**.
4. **Data Acquisition & Processing Loop (Runs every 2 seconds):**
 - Reads raw values from DHT22 and DS18B20.
 - Applies calibration inversion equations: $y = 1.0318x - 0.6398$.
 - - **Calibrated DHT22 = Raw_DHT**
 - **Calibrated DS18B20 = (Raw_DS + 0.6398) / 1.0318**
 - Computes fused temperature using the Sequential Kalman Filter.
 - Reads and parses latitude/longitude coordinates from the NEO-6M GPS.
 - Transmits telemetry payloads over MQTT.

MQTT Interface & Command Reference

Published Telemetry Topics (ESP32 -> Broker)

MQTT Topic	Payload Format	Description	Example Payload
EE2120/ESP01/temp	String (Float)	Calibrated and Kalman-fused temperature (°C)	25.43
EE2120/ESP01/gps	String (CSV)	Latitude and Longitude coordinates	7.2450, 80.6850
EE2120/ESP01/LED/status	String (0 or 1)	Execution state of the built-in LED actuator	1

Subscribed Command Topics (Broker -> ESP32)

MQTT Topic	Valid Commands	Action Triggered
EE2120/ESP01/LED/cmd	1	Turns ON built-in LED and publishes 1 to .../LED/status
	0	Turns OFF built-in LED and publishes 0 to .../LED/status

Troubleshooting & LED Indicators

- **Serial Output Displays "Sensor Error" :**
 - Verify 3.3V power supply connections to the DHT22 and DS18B20.
 - Check for presence of pull-up resistors on GPIO 15 (10k ohm) and GPIO 4 (4.7k ohm).
- **MQTT Reconnection Loops (failed, rc=-2 or rc=-4):**
 - Check if the MQTT broker computer (192.168.8.141) is online and pingable from the network.
 - Ensure port 1883 is allowed through the broker machine's firewall.
 - Verify username (esp01) and password (stud1) credentials.
- **Invalid or Blank GPS Data:**
 - Ensure the NEO-6M module has line-of-sight to outdoor skies.
 - Check if the red LED on the NEO-6M is blinking (blinking indicates a valid satellite fix; solid/off means searching for satellites).
 - Confirm RX2 and TX2 wiring is swapped correctly (ESP32 TX2 -> GPS RX, ESP32 RX2 -> GPS TX).

Distributed Temperature Monitoring System: Implementation Summary

